

Хеширование и безопасное хранение паролей

Argon2id, salt, стоимость вычисления и безопасный passwordService

Результаты занятия

- ✓ Объяснять, почему пароль нельзя хранить в открытом виде
- ✓ Различать hashing и encryption
- ✓ Понимать назначение salt
- ✓ Объяснять, почему быстрые hash-функции не подходят для password storage
- ✓ Понимать преимущества memory-hard алгоритмов
- ✓ Использовать Argon2id для password hashing
- ✓ Использовать `argon2.hash()` и `argon2.verify()`
- ✓ Понимать `memoryCost`, `timeCost` и `parallelism`
- ✓ Интегрировать `passwordService` в `register/login`
- ✓ Не возвращать `passwordHash` клиенту

Учебные вопросы

- | | | |
|---|---------------------------------------|---|
| 1 | Почему пароль нужно хешировать | Угроза утечки базы, hashing vs encryption и fast hash |
| 2 | Salt и структура password hash | Уникальный salt и закодированная строка Argon2 |
| 3 | Argon2id | Memory-hard hashing и параметры стоимости |
| 4 | hash() и verify() в Node.js | Практическая реализация passwordService |
| 5 | Интеграция и эксплуатация | Register/login, rehash, ошибки и правила безопасности |



Учебный вопрос 1. Почему пароль нужно хешировать

Утечка базы не должна раскрывать исходные passwords

Главная угроза: компрометация базы

- База данных может быть украдена из резервной копии, SQL-инъекции или скомпрометированного сервера.
- Если password хранится открытым текстом, злоумышленник получает его сразу.
- Пользователи часто повторно используют passwords на разных сервисах.
- Поэтому даже администратор базы не должен иметь возможность прочитать исходный password.
- Password storage проектируется с предположением, что hashes однажды могут попасть к атакующему.

Hashing и encryption

Hashing

- Одностороннее преобразование
- Исходное значение не требуется восстанавливать
- Подходит для проверки password
- При login выполняется verify

Encryption

- Обратимое преобразование
- Данные можно расшифровать при наличии ключа
- Подходит для данных, которые нужно восстановить
- Для password storage обычно не используется

Что хранить в users

Поле	Хранить?	Причина
password	Нет	Исходный секрет нельзя сохранять
passwordHash	Да	Используется для проверки password
salt отдельно	Обычно не требуется	Современный Argon2 encoding включает salt
алгоритм и параметры	Да, внутри encoded hash	Нужны для последующей verify

Plaintext vs Encryption vs Password Hashing

Три подхода к хранению паролей: почему хеширование — правильный выбор

Безопасность
в веб-приложениях

1 Plaintext

Хранение пароля в открытом виде

Регистрация / Login

MyP@ssw0rd

Отправить



users

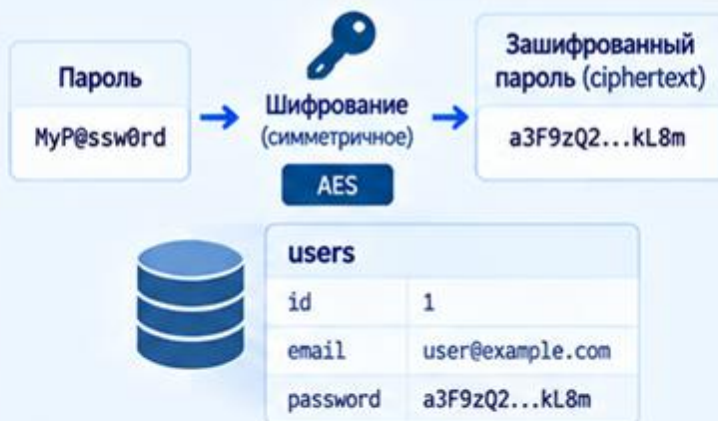
id	1
email	user@example.com
password	MyP@ssw0rd

- ✗ Пароль хранится как есть (plaintext)
- ✗ При утечке базы все пароли сразу скомпрометированы
- ✗ Полный доступ к аккаунтам пользователей

✗ **Никогда не используйте!**
Это небезопасно

2 Encryption

Двустороннее шифрование (обратимо)



- ✓ Пароль хранится в зашифрованном виде
- ✓ Можно расшифровать и получить исходный пароль (при наличии ключа)
- ✗ Если ключ скомпрометирован — все пароли могут быть расшифрованы

! **Не рекомендуется для хранения паролей.**
Подходит для обратимо шифруемых данных

3 Password Hashing

Одностороннее хеширование (необратимо)



- ✓ Пароль преобразуется в хеш (необратимо)
- ✓ Для одного и того же пароля всегда получается разный хеш (из-за соли)
- ✓ Даже при утечке базы восстановить исходные пароли практически невозможно

✓ **Рекомендуемый подход!**
Используйте современные алгоритмы хеширования

Почему SHA-256 недостаточно

- SHA-256 разработан как быстрый криптографический hash.
- Для файлов и цифровых подписей скорость полезна.
- Для password storage высокая скорость помогает атакующему проверять огромное количество guesses.
- Password hashing должен быть намеренно дорогим по CPU и памяти.
- Поэтому используются Argon2id, scrypt, bcrypt или PBKDF2, а не обычный SHA-256.

Offline password cracking

- После кражи базы атакующий может проверять guesses локально.
- Сервер уже не может применить rate limit к этим попыткам.
- Защита зависит от сложности password и стоимости одного вычисления hash.
- Memory-hard алгоритмы увеличивают стоимость массового перебора на GPU и специализированном оборудовании.



Учебный вопрос 2. Salt и структура password hash

Одинаковые passwords не должны давать одинаковые сохранённые значения

Что такое salt

- Salt — случайное уникальное значение, которое участвует в password hashing.
- Для каждой учётной записи используется собственный salt.
- Salt не является секретом.
- Salt хранится вместе с hash.
- Одинаковый password с разными salts даёт разные encoded hashes.
- Salt защищает от заранее рассчитанных rainbow tables.

Одинаковый password — разные hashes

Запрос: Два пользователя выбирают один password

```
User A: password = Example123!  
User B: password = Example123!
```

Результат

```
Hash A:  
$argon2id$...$saltA$...  
Hash B:  
$argon2id$...$saltB$...
```

Пояснение

- Уникальный salt делает сохранённые значения различными.

Encoded hash содержит метаданные

- Современный Argon2 hash обычно хранится в PHC-подобной строке.
- Строка содержит вариант алгоритма.
- Содержит версию и параметры стоимости.
- Содержит salt.
- Содержит итоговый hash.
- Благодаря этому verify знает, как проверить password.

Пример Argon2id encoded string

```
$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>
```

Salt: одинаковый password → разные Argon2id hashes



Ключевые выводы:

- Salt — случайное уникальное значение для каждого password hash.
- Даже одинаковые passwords не дают одинаковый сохранённый hash.
- Salt хранится вместе с hash и не является секретом.
- Это защищает от rainbow tables и упрощённого массового сравнения hashes.



Данные в базе (например, таблица users)

id	username	passwordHash
1	user_a	\$argon2id\$v=19\$m=65536,t=3,p=4\$f3e9a7c2b1d4e8f9\$K7xQ9mZt...
2	user_b	\$argon2id\$v=19\$m=65536,t=3,p=4\$9c6d2f8e4a1b7c3d\$W4rT8pLm...

В базе хранятся уже готовые хэши (вместе с salt и параметрами Argon2id).



Учебный вопрос 3. Argon2id

Memory-hard password hashing для современных приложений

Почему Argon2id

- Argon2id — вариант Argon2, предназначенный для password hashing и сочетающий защитные свойства Argon2i и Argon2d.
- Argon2 победил Password Hashing Competition.
- Argon2id устойчив к массовому перебору на GPU благодаря затратам памяти.
- OWASP рекомендует Argon2id для новых приложений.
- Стоимость вычисления можно адаптировать к возможностям сервера.

Argon2: три основных параметра

Параметр	Что регулирует	Эффект
memoryCost	Объём используемой памяти	Увеличивает стоимость атаки по памяти
timeCost	Количество проходов/итераций	Увеличивает время вычисления
parallelism	Степень параллельности	Определяет число lanes/параллельных вычислений

Memory-hard — ключевая идея

- Обычный быстрый hash в основном требует вычислительной мощности.
- Argon2 дополнительно требует значительный объём памяти.
- При массовом переборе память становится дорогим ограничивающим ресурсом.
- Цель — сделать один login приемлемым для сервера, но миллионы guesses дорогими для атакующего.

Ориентир OWASP для Argon2id

memoryCost	timeCost	parallelism	Комментарий
19 MiB	2	1	Минимальная рекомендуемая конфигурация
46 MiB	1	1	Альтернативный баланс CPU/RAM
12 MiB	3	1	Больше итераций, меньше памяти

Не максимизируйте параметры бездумно

- Слишком дешёвый hash облегчает brute force.
- Слишком дорогой hash замедляет login легитимных пользователей.
- Очень тяжёлый hash может усилить DoS-риск при массовых запросах login.
- Параметры выбирают на реальном сервере и периодически пересматривают.
- OWASP рекомендует держать вычисление hash практически приемлемым для приложения.

Argon2id: password + salt + memory/time cost → encoded hash



Salt делает одинаковые пароли разными.



memoryCost и **timeCost** повышают стоимость перебора.



encoded hash хранит алгоритм, параметры, salt и hash.



Для проверки используется **verify()**.



Учебный вопрос 4. `hash()` и `verify()` в Node.js

Пакет `argon2` скрывает низкоуровневую криптографию

Пакет `argon2` для Node.js

- Пакет предоставляет `async hash()` и `verify()`.
- `Argon2id` используется по умолчанию.
- `Salt` генерируется библиотекой автоматически.
- `hash()` возвращает `encoded string`.
- `verify()` получает `encoded hash` и введённый `password`.

Установка

```
npm install argon2
```

Хеширование password

JAVASCRIPT

```
import * as argon2 from "argon2";  
  
const passwordHash = await argon2.hash(password);
```

Что важно

- password существует в памяти приложения только во время обработки запроса.
- argon2.hash() генерирует salt автоматически.
- Результат содержит алгоритм, параметры, salt и hash.
- В PostgreSQL сохраняется passwordHash.

Проверка password

JAVASCRIPT

```
const valid = await argon2.verify(  
  user.passwordHash,  
  password  
);  
  
if (!valid) {  
  return null;  
}
```

Что важно

- Сначала загружается сохранённый passwordHash.
- verify() извлекает параметры и salt из encoded string.
- Введённый password проверяется относительно сохранённого значения.
- Исходный password из hash не восстанавливается.

argon2.hash() при register и argon2.verify() при login

Безопасные
пароли —
более безопасный
мир

Register



Login



Login не создаёт новый hash для сравнения



При login используется verify(), а не новое hash()

Проверяем,
а не пересчитываем!

Безопасность
начинается
с правильных
принципов

Один пароль.
Два сценария.
Одна цель —
безопасность.

passwordService.js

JAVASCRIPT

```
import * as argon2 from "argon2";

export async function hashPassword(password) {
  return argon2.hash(password);
}

export async function verifyPassword(passwordHash, password) {
  return argon2.verify(passwordHash, password);
}
```

Что важно

- Криптографическая логика изолирована в отдельном Service.
- Auth Service не зависит от деталей библиотеки.
- Позже алгоритм или параметры можно изменить централизованно.

Почему не нужно вручную генерировать salt

- Современная password-hashing библиотека должна корректно генерировать случайный salt.
- Ручная генерация salt добавляет лишний код и риск ошибки.
- node-argon2 сам формирует encoded hash со всеми нужными данными.
- В таблице users достаточно одного поля passwordHash.

Что нельзя делать с hash

- Не сравнивайте passwordHash через обычное ===.
- Не хешируйте введённый password заново и не сравнивайте две строки hash.
- Новый вызов hash() использует новый salt и создаёт другое значение.
- Для проверки всегда используйте verify(storedHash, password).



Учебный вопрос 5. Интеграция и эксплуатация

Password hashing становится частью register/login, но не всего приложения

Register Service

JAVASCRIPT

```
export async function registerUser({ name, email, password }) {  
  const passwordHash = await hashPassword(password);  
  
  return prisma.user.create({  
    data: { name, email, passwordHash },  
    select: { id: true, name: true, email: true, role: true }  
  });  
}
```

Что важно

- password приходит из HTTP body.
- Перед записью вызывается hashPassword().
- В Prisma передаётся passwordHash.
- select исключает passwordHash из ответа.

Login Service

JAVASCRIPT

```
export async function authenticateUser(email, password) {
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return null;

  const valid = await verifyPassword(user.passwordHash, password);
  if (!valid) return null;

  return {
    id: user.id,
    name: user.name,
    email: user.email,
    role: user.role
  };
}
```

Что важно

- Login не создаёт новый hash.
- Используется verify() относительно сохранённого encoded hash.
- И неизвестный email, и неверный password дают одинаковый внешний результат.
- passwordHash остаётся только на сервере.

Что хранится после регистрации

Запрос: POST /api/auth/register

```
{  
  "email": "alex@example.com",  
  "password": "ExamplePassword123!"  
}
```

Результат

```
users.password_hash =  
$argon2id$...  
HTTP response = { id, name,  
email, role }
```

Пояснение

- Исходный password не сохраняется и не возвращается.

Полный путь password: register → hash → DB → login → verify

От регистрации пользователя до последующей проверки пароля. Plaintext пароль нигде не хранится.

Безопасное хранение и проверка паролей в Express Internet Shop

1

Register (регистрация)

Пользователь создаёт аккаунт

Создание аккаунта

user@example.com

••••••••

Зарегистрироваться

2

Auth Controller

Принимает данные (email, password)

```
// authController.register()
const { email, password } = req.body;
await authService.register({
  email,
  password
});
```

3

Auth Service

Хеширует пароль

```
// authService.register()
const passwordHash = await
  argon2.hash(password, {
    type: argon2.argon2id,
    memoryCost: 2 ** 16,
    timeCost: 3,
    parallelism: 1
  });
await prisma.user.create({
  data: { email, passwordHash }
});
```

4

Prisma Client

Сохраняет в базу

```
// Prisma
await prisma.user.create({
  data: {
    email: "user@example.com",
    passwordHash
  }
});
```

5

PostgreSQL

В базе хранится хеш



users

id	UUID
email	VARCHAR
passwordHash	VARCHAR
name	VARCHAR
role	VARCHAR
createdAt	TIMESTAMP



Пользователь успешно зарегистрирован!

В базе хранится только хеш пароля, а не сам пароль.

1

Login (вход)

Пользователь вводит свои данные

Вход в аккаунт

user@example.com

••••••••

Войти

Нет аккаунта? Регистрация

2

Auth Controller

Передаёт данные в сервис

```
// authController.login()
const { email, password } = req.body;
const user = await authService.login({
  email,
  password
});
```

3

Auth Service

Проверяет пароль

```
// authService.login()
const user = await prisma.user
  .findUnique({ where: { email } });
if (!user) throw new Error('Invalid');
const isValid = await argon2.verify(
  user.passwordHash,
  password
);
if (!isValid) throw new Error('Invalid');
return user;
```

4

Prisma Client

Ищет пользователя

```
// Prisma
const user = await prisma.user
  .findUnique({
    where: { email },
    select: {
      id: true,
      email: true,
      passwordHash: true,
      name: true,
      role: true
    }
  });
```

5

Сравнение хеша

argon2.verify()

Введённый пароль (plaintext)

MyP@ssw0rd

Stored passwordHash (из базы)

\$argon2id\$v=19\$m=...\$



true

Пароль верный
Пользователь авторизован



false

Неверный пароль
Возвращается ошибка 401



При входе НЕ создаётся новый хеш!
Введённый пароль проверяется через argon2.verify() с сохранённым хешем.



Ключевые принципы

- ✓ Plaintext пароль нигде не хранится
- ✓ Используйте Argon2id (рекомендуется OWASP)
- ✓ Уникальный salt генерируется автоматически
- ✓ Настройте параметры (memory, time, parallelism)



Рекомендуемые параметры Argon2id

- memoryCost: 2 ** 16 (64 MiB)
- timeCost: 2-3
- parallelism: 1
- Используйте библиотеку "argon2" для Node.js



Результат

- ✓ Безопасное хранение паролей
- ✓ Устойчивая защита от перебора (brute force)
- ✓ Соответствие современным рекомендациям (OWASP)
- ✓ Готово к использованию в Express Internet Shop



Express
Internet Shop

Безопасно.
Современно.
Надёжно.

Изменение параметров со временем

- Аппаратное обеспечение становится быстрее.
- Стоимость password hashing следует периодически пересматривать.
- Encoded hash хранит параметры старого вычисления.
- После успешного login старый hash можно пересчитать с более сильными параметрами.
- Обновление можно выполнять постепенно при входе пользователей.

Pepper — дополнительный слой

- Pepper — общий секрет приложения, дополнительный к уникальному salt.
- Pepper не хранится рядом с password hashes в базе.
- Его следует хранить в secrets manager или защищённой конфигурации.
- Pepper усложняет эксплуатацию и ротацию.
- Для учебного Internet Shop pepper не требуется.

Password policy и hashing — разные задачи

- Hashing защищает сохранённые credentials.
- Password policy влияет на устойчивость самого password к guessing.
- Не следует полагаться только на сложность алгоритма.
- Rate limiting защищает online login attempts.
- MFA дополнительно снижает риск компрометации одного password.

Правила реализации

- ✓ Никогда не хранить plaintext password
- ✓ Не использовать MD5, SHA-1 или простой SHA-256 для password storage
- ✓ Использовать специализированный password hashing
- ✓ Для новых приложений предпочитать Argon2id
- ✓ Доверять библиотеке генерацию salt
- ✓ Использовать `verify()`, а не ручное сравнение hashes
- ✓ Не возвращать passwordHash клиенту
- ✓ Не писать password и passwordHash в обычные application logs
- ✓ Использовать HTTPS для передачи credentials

Типичные ошибки студентов

Ошибка	Почему плохо	Правильно
Сохранить password	Утечка раскрывает credentials	Сохранять passwordHash
SHA-256(password)	Слишком быстрый для password storage	Argon2id
Один общий salt	Одинаковые passwords легче сопоставлять	Уникальный salt автоматически
hash(password) === storedHash	Новый salt создаёт новый hash	argon2.verify()
Вернуть User целиком	Может утечь passwordHash	Использовать select

Производительность login

- Password verification намеренно дороже обычных операций.
- Это нормально для authentication endpoint.
- Не следует выполнять Argon2 для каждого обычного API-запроса.
- После login пользователь идентифицируется через session или token.
- Rate limiting помогает защитить login endpoint от массового перебора.

Мини-практика: passwordService

Проверьте, как Argon2id создаёт и проверяет hashes.

Задание

- Создать два hash одного password
- Сравнить строки и убедиться, что они различаются
- Проверить оба hash через verify()
- Проверить неправильный password
- Рассмотреть encoded string и найти обозначение argon2id

Критерии проверки

- Два hash одного password различаются
- verify() возвращает true для правильного password
- verify() возвращает false для неправильного password
- Студент объясняет роль salt

Небольшой эксперимент

JAVASCRIPT

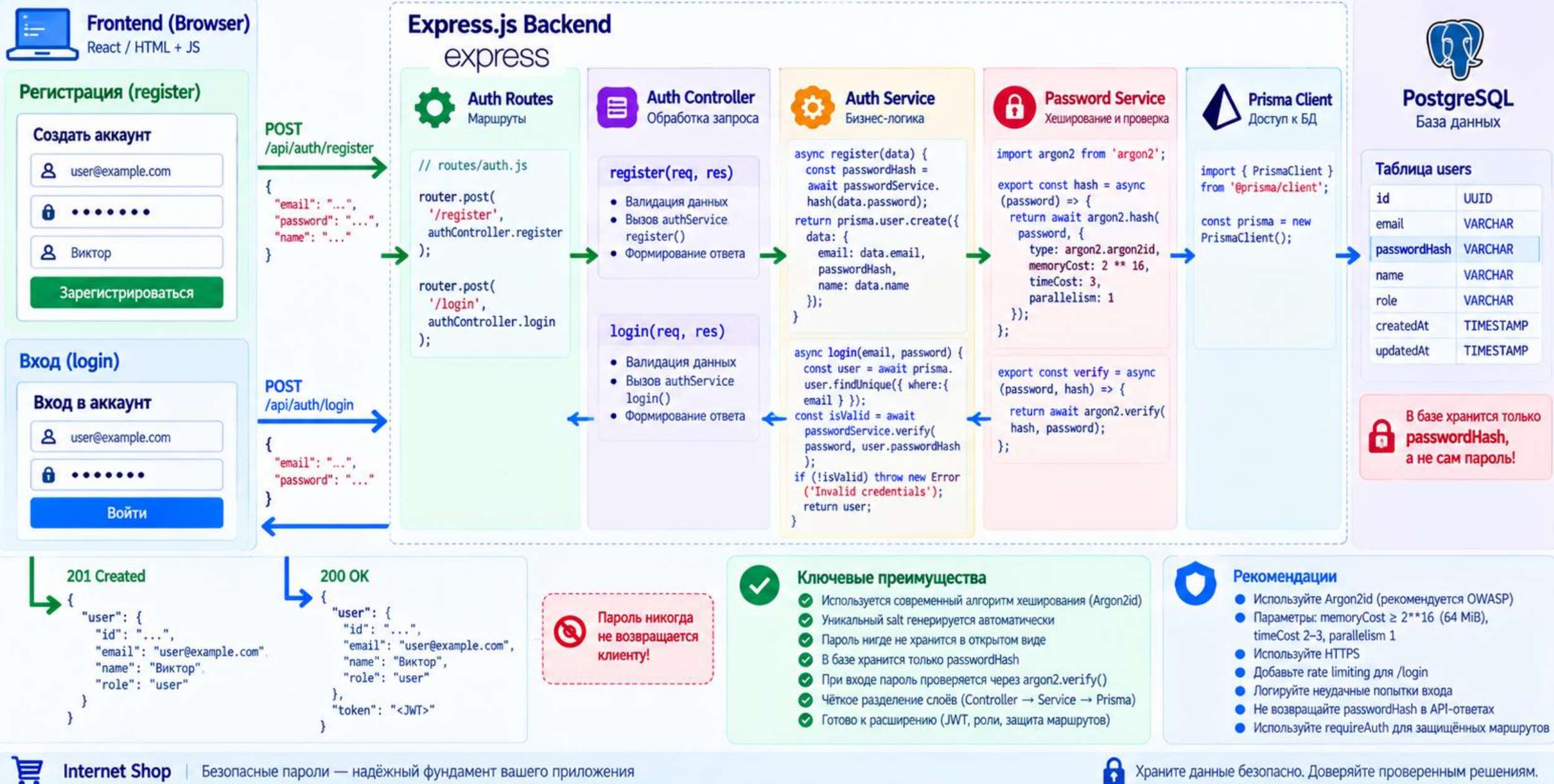
```
const hash1 = await argon2.hash("Example123!");  
const hash2 = await argon2.hash("Example123!");  
  
console.log(hash1 === hash2); // false  
console.log(await argon2.verify(hash1, "Example123!")); // true  
console.log(await argon2.verify(hash1, "wrong")); // false
```

Что важно

- Одинаковый password даёт разные encoded hashes из-за разных salts.
- Различие hashes не мешает verify().
- Это ожидаемое и правильное поведение.

Архитектура безопасного password storage в Internet Shop

Разделение ответственности, безопасное хеширование паролей и защита данных пользователей



Контрольные вопросы

1. Почему password нельзя хранить открытым текстом?
2. Чем hashing отличается от encryption?
3. Почему SHA-256 не подходит для password storage?
4. Что такое salt?
5. Нужно ли скрывать salt?
6. Почему два одинаковых passwords дают разные Argon2 hashes?
7. Что регулируют memoryCost и timeCost?
8. Для чего используется argon2.verify()?
9. Почему нельзя сравнивать новый hash password со storedHash через ===?
10. Почему passwordHash нельзя отправлять frontend?

Итоги занятия

1

Passwords должны храниться только в виде специализированных password hashes.

2

Hashing является односторонним, в отличие от encryption.

3

Быстрые hash-функции не подходят для password storage.

4

Уникальный salt защищает от precomputed attacks и одинаковых hashes.

5

Argon2id — современный memory-hard алгоритм для password hashing.

6

node-argon2 использует Argon2id по умолчанию и автоматически создаёт salt.

7

hash() используется при регистрации и смене password.

8

verify() используется при login.

9

passwordService изолирует криптографическую логику.

10

passwordHash никогда не должен попадать в клиентский ответ.